

*Please come back
to your desk!*

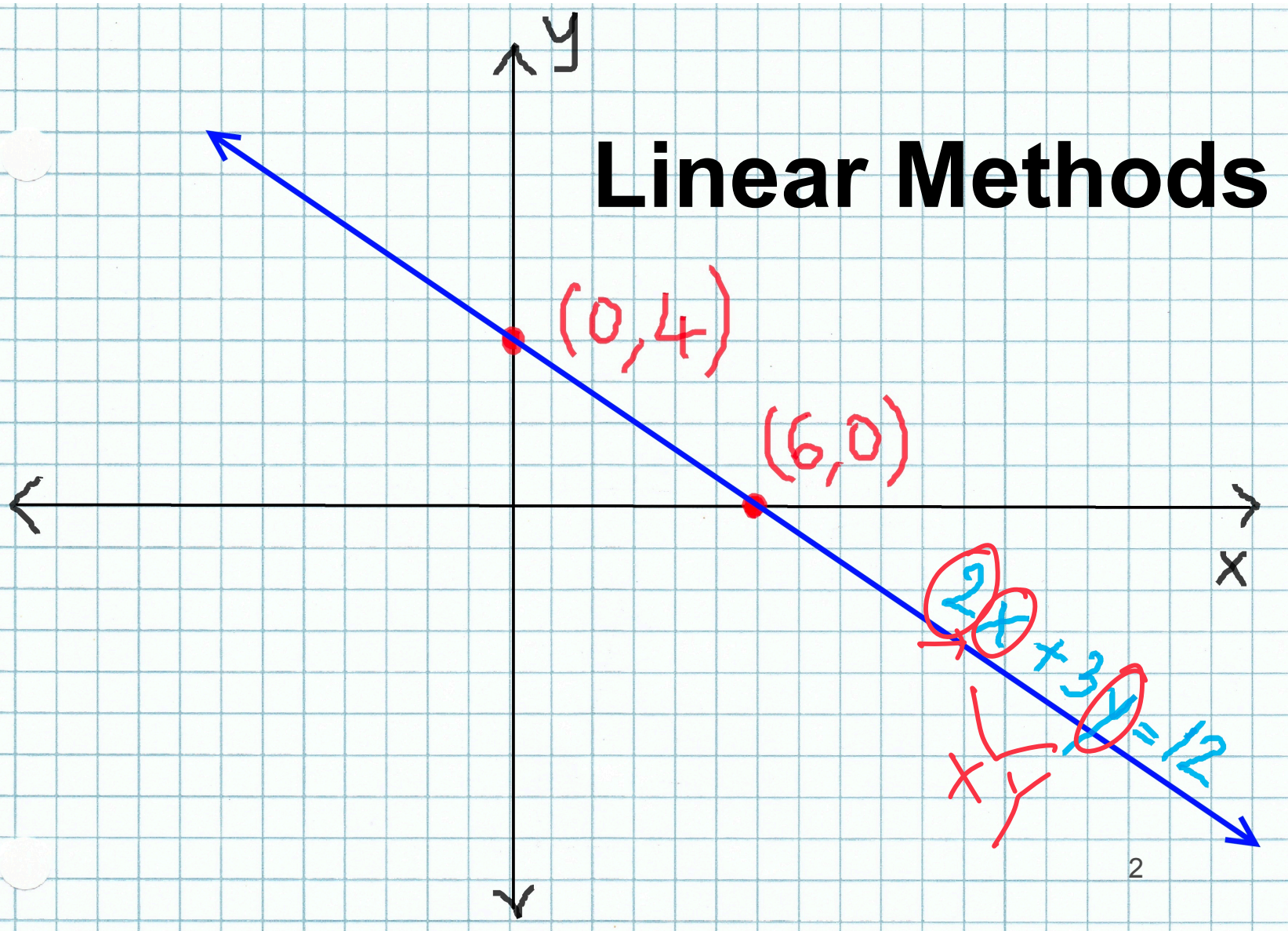
Introduction to Deep Learning

4. Linear Regression, Basic Optimization

MGMT 735 DEEP LEARNING IN FINANCE

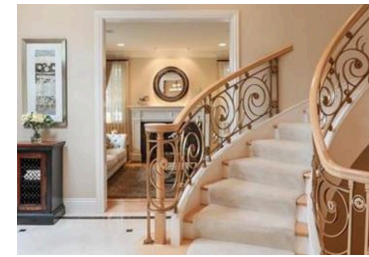
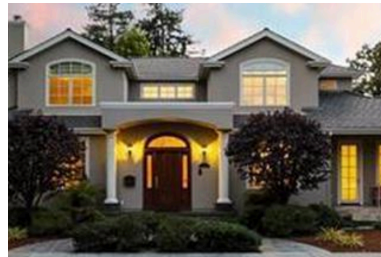
SLIDES FROM Alex Smola and Mu Li

Linear Methods



House Buying 101

- Pick a house, take a tour, and read facts
- Estimate its price, bid



Listing price
from agent

\$5,498,000

Price

7

Beds

5

Baths

4,865 Sq. Ft.

\$1130 / Sq. Ft.

Redfin Estimate: \$5,390,037 On Redfin: 15 days

Predicted
sale price

Virtual Tour

- [Branded Virtual Tour](#)
- [Virtual Tour \(External Link\)](#)

Parking Information

- Garage (Minimum): 2
- Garage (Maximum): 2
- Parking Description: Attached Garage, On Street
- Garage Spaces: 2

Interior Features

Bedroom Information

- # of Bedrooms (Minimum): 7
- # of Bedrooms (Maximum): 7

3

Multi-Unit Information

- # of Stories: 2

School Information

- Elementary School: El Carmelo El
- Elementary School District: Palo A
- Middle School: Jane Lathrop Stan
- High School: Palo Alto High
- High School District: Palo Alto Un

- Kitchen Description: Countertop, Dishwasher, Garbage Disposal, Island with Sink, Microwave, Over

A Simplified Model

7 Beds	5 Baths	4,865 Sq. Ft. \$1130 / Sq. Ft.
Estimate: \$5,390,037 On Redfin: 15 days		

- Assumption 1 *attributes / variable predictors*
The key factors impacting the prices are
#Beds, #Baths, Living Sqft, denoted by x_1, x_2, x_3

- Assumption 2
The sale price is a weighted sum over the key factors

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \underline{b}$$

Weights and bias are determined later

Linear Model

- Given n -dimensional inputs $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$
- Linear model has a n -dimensional weight and a bias

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T, \quad b$$

- The output is a weighted sum of the inputs

$$y = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

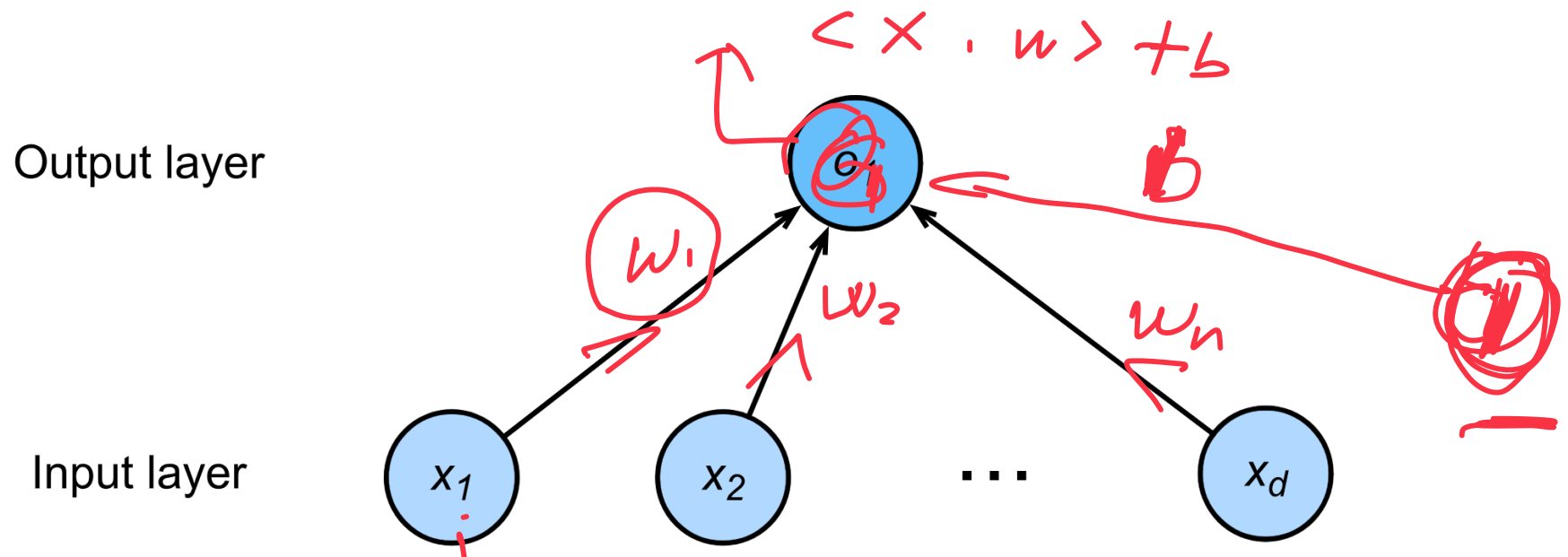
Vectorized version

$$y = \langle \mathbf{w}, \mathbf{x} \rangle + b$$

~~total~~ row vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

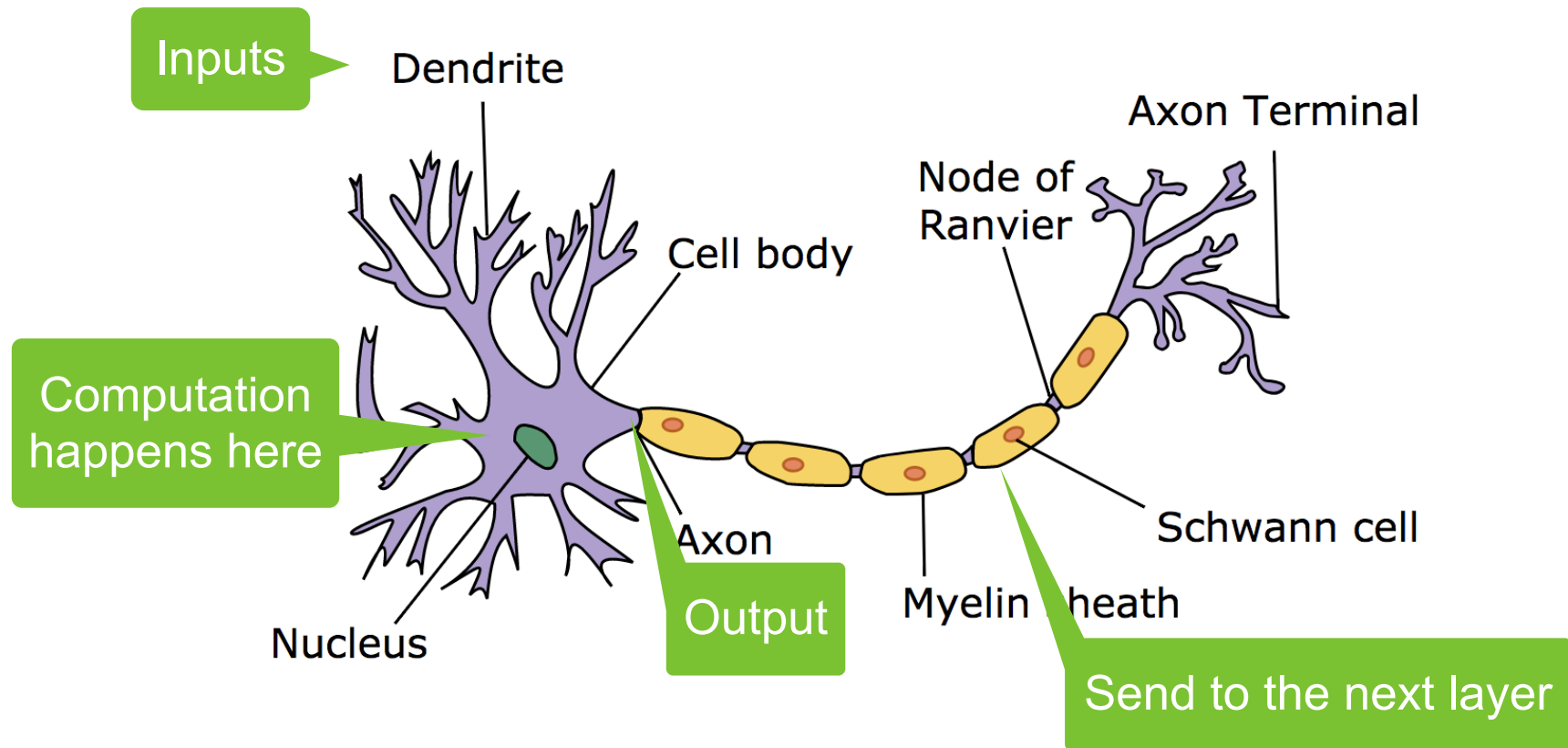
Linear Model as a Single-layer Neural Network



We can stack multiple layers to get deep neural networks

Neural Networks Derive from Neuroscience

The real neuron



Measure Estimation Quality

- Compare the true value vs the estimated value
Real sale price vs estimated house price
- Let y the true value, and $\hat{y}$ the estimated value, we can compare the loss

$$\ell(y, \hat{y}) = (y - \hat{y})^2 \quad \textcircled{0_1}$$

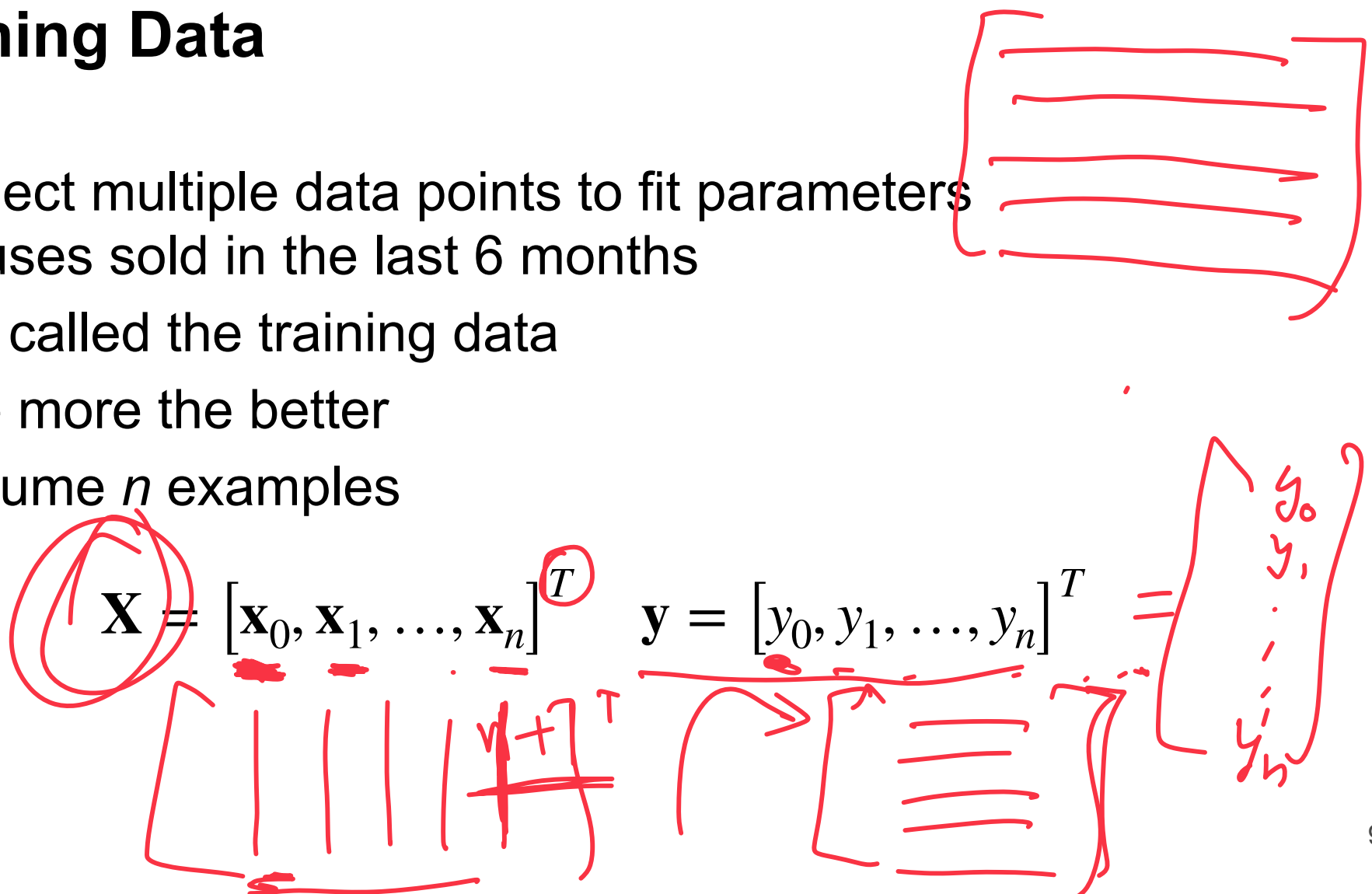
It is called squared loss

n inputs. $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

MSE

Training Data

- Collect multiple data points to fit parameters
Houses sold in the last 6 months
- It is called the training data
- The more the better
- Assume n examples


$$\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_n]^T \quad \mathbf{y} = [y_0, y_1, \dots, y_n]^T$$

Learn Parameters

- Training loss

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y_i - \langle \mathbf{x}_i, \mathbf{w} \rangle - b)^2 = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\mathbf{w} - b\|^2$$

Handwritten red annotations: $\hat{y}_i$ above y_i ; $\hat{\mathbf{y}} = \begin{pmatrix} \hat{y}_0 \\ \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{pmatrix}$ and $\begin{pmatrix} b \\ b \\ b \\ \vdots \\ b \end{pmatrix}$ to the right of the equation.

- Minimize loss to learn parameters

$$\underline{\mathbf{w}^*}, \underline{\mathbf{b}^*} = \arg \min_{\underline{\mathbf{w}}, \underline{b}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}, b)$$

Handwritten red annotations: A large circle around the entire equation; a bracket under $\arg \min$; a bracket under $\mathbf{w}, b$; and a vertical line with a dot at the bottom right.

Closed-form Solution

- Add bias into weights by $\mathbf{X} \leftarrow [\mathbf{X}, \mathbf{1}]$ $\mathbf{w} \leftarrow \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 \quad \frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = \frac{2}{n} (\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{X}$$

- Loss is convex, so the optimal solutions satisfies

$$\frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = 0$$

$$\frac{\partial}{\partial \mathbf{w}} \ell(\mathbf{X}, \mathbf{y}, \mathbf{w}) = 0 \Leftrightarrow \frac{2}{n} (\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{X} = 0$$

$$\Leftrightarrow \mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$



Handwritten red text: $\frac{\partial}{\partial \mathbf{w}}$

Handwritten red text: $\frac{2}{n}$

Example on how to calculate the gradient

Assume $\mathbf{X} \in \mathbb{R}^{m \times n}$, $\mathbf{w} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

$$z = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

Compute $\frac{\partial z}{\partial \mathbf{w}}$

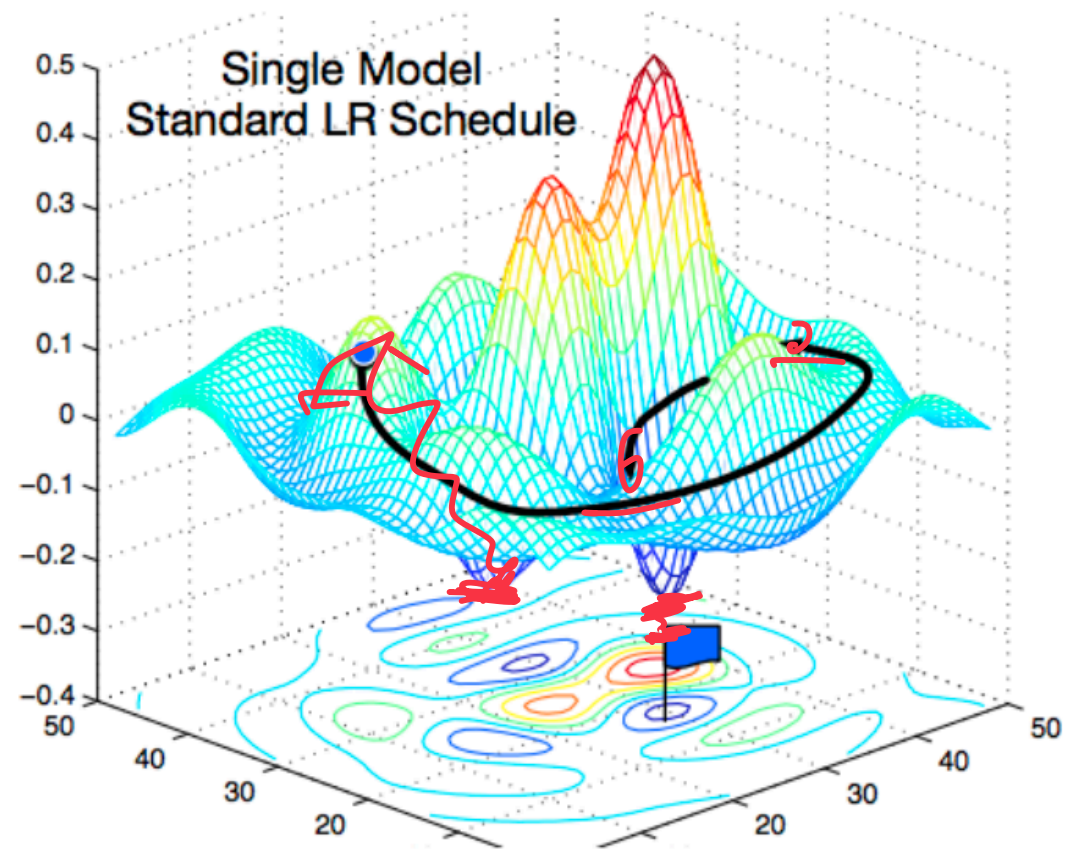
$$\begin{aligned} \frac{\partial z}{\partial \mathbf{w}} &= \frac{\partial z}{\partial \mathbf{b}} \frac{\partial \mathbf{b}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{w}} \\ &= \frac{\partial \|\mathbf{b}\|^2}{\partial \mathbf{b}} \frac{\partial (\mathbf{a} - \mathbf{y})}{\partial \mathbf{a}} \frac{\partial \mathbf{X}\mathbf{w}}{\partial \mathbf{w}} \end{aligned}$$

Decompose

$$\begin{aligned} \mathbf{a} &= \mathbf{X}\mathbf{w} \\ \mathbf{b} &= \mathbf{a} - \mathbf{y} \\ z &= \|\mathbf{b}\|^2 \end{aligned}$$

$$\begin{aligned} &= 2\mathbf{b}^T \times \mathbf{I} \times \mathbf{X} \\ &= 2(\mathbf{X}\mathbf{w} - \mathbf{y})^T \mathbf{X} \end{aligned}$$

Basic Optimization



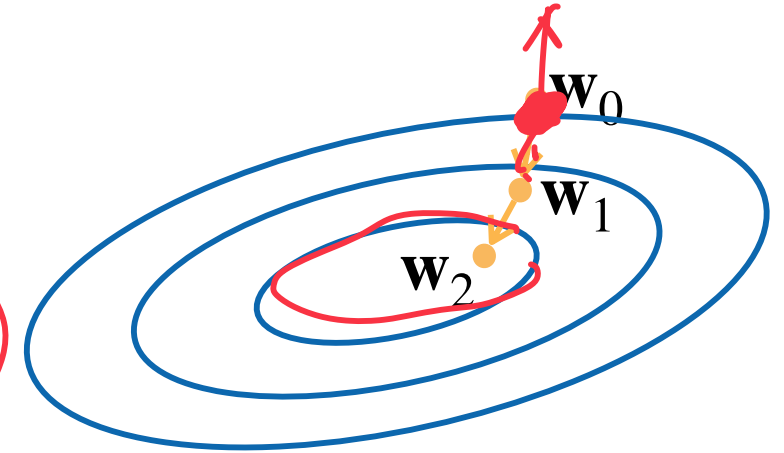
Gradient Descent

- Choose a starting point $\mathbf{w}_0$
- Repeat to update the weight $t=1,2,3$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$$

(Handwritten red annotations: underlines $\mathbf{w}_{t-1}$, η , and $\frac{\partial \ell}{\partial \mathbf{w}_{t-1}}$; a red arrow points from η to the loss function $\ell(\mathbf{w}_t)$ in the adjacent diagram.)

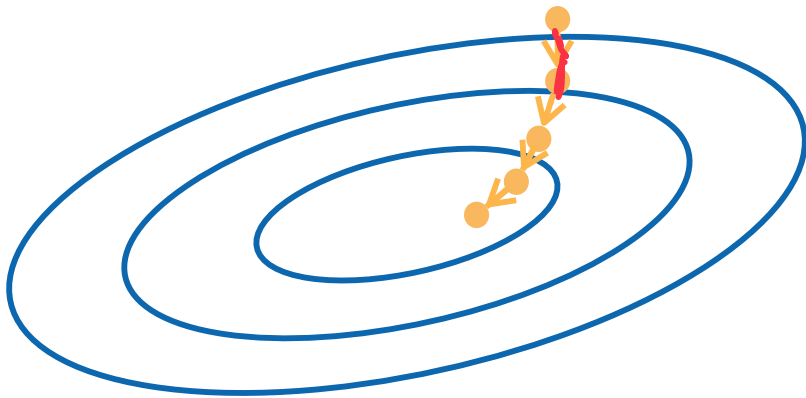
$\ell(\mathbf{w}_t)$



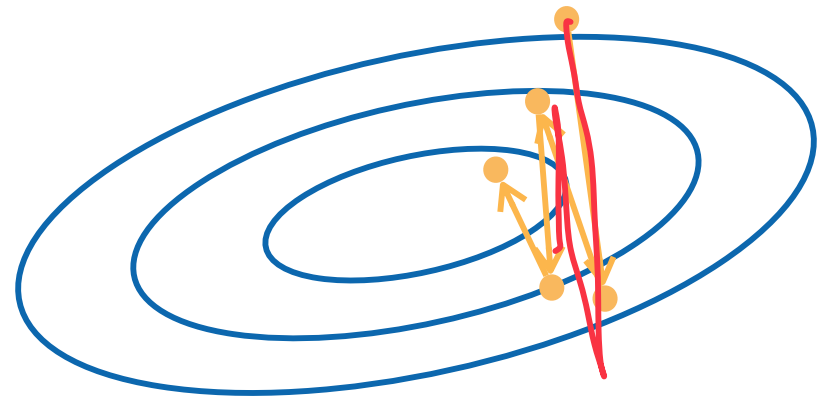
- Gradient: a direction that increases the value
- Learning rate: a hyper-parameter specifies the step length

Choose a Learning Rate

Not too small



Not too big



Mini-batch Stochastic Gradient Descent (SGD)

- Computing the gradient over the whole training data is too expensive
 - Takes minutes to hours for DNN models
- Randomly sample b examples $i_1, i_2, \dots, i_b$ to approximate the loss

$$\frac{1}{b} \sum_{i \in I_b} \ell(\mathbf{x}_i, y_i, \mathbf{w})$$

$b = 32$
64
128

- b is the batch size, another important hyper-parameters

256
512

Choose a Batch Size

Not too small

Workload is too small,
hard to fully utilize
computation resources

Not too big

Memory issue
Waste computation, e.g.
when all x_i are identical

Summary

- Problem: estimate a real value
- Model: $y = \langle \mathbf{w}, \mathbf{x} \rangle + b$
- Loss: square loss $\ell(y, \hat{y}) = (y - \hat{y})^2$
- Learn by mini-batch SGD
 - Choose a starting point
 - Repeat
 - Compute gradient
 - Update parameters



Regression vs. Classification

- Regression estimates a continuous value
- Classification predicts a discrete category

MNIST: classify hand-written digits
(10 classes)

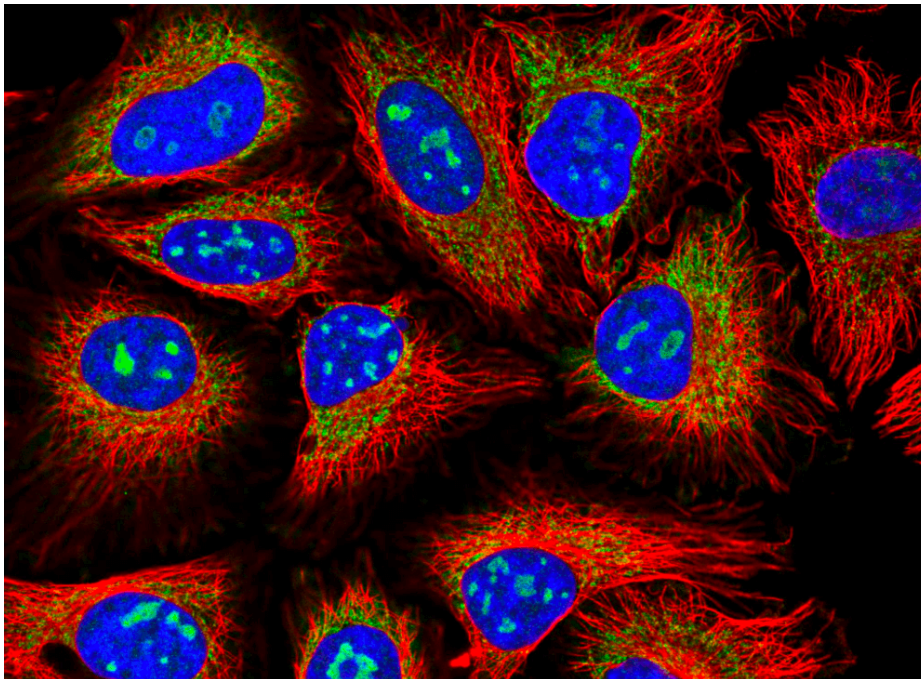


ImageNet: classify nature objects
(1000 classes)



Classification Tasks at Kaggle

Classify human protein microscope images into 28 categories



- 0. Nucleoplasm
- 1. Nuclear membrane
- 2. Nucleoli
- 3. Nucleoli fibrillar
- 4. Nuclear speckles
- 5. Nuclear bodies
- 6. Endoplasmic reticu
- 7. Golgi apparatus
- 8. Peroxisomes
- 9. Endosomes
- 10. Lysosomes
- 11. Intermediate fila
- 12. Actin filaments
- 13. Focal adhesion si
- 14. Microtubules
- 15. Microtubule ends
- 16. Cytokinetic bridg

<https://www.kaggle.com/c/human-protein-atlas-image-classification>

Classification Tasks at Kaggle

Classify malware into 9 categories

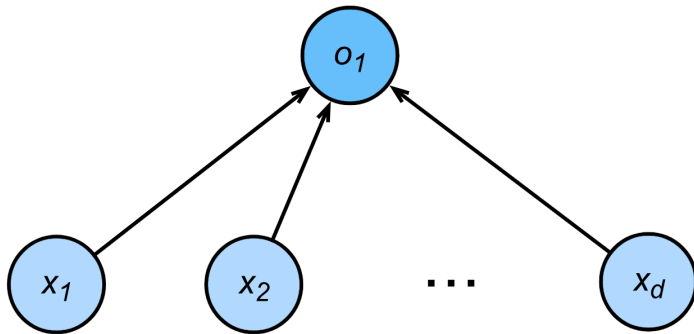


<https://www.kaggle.com/c/malware-classification>

From Regression to Multi-class Classification

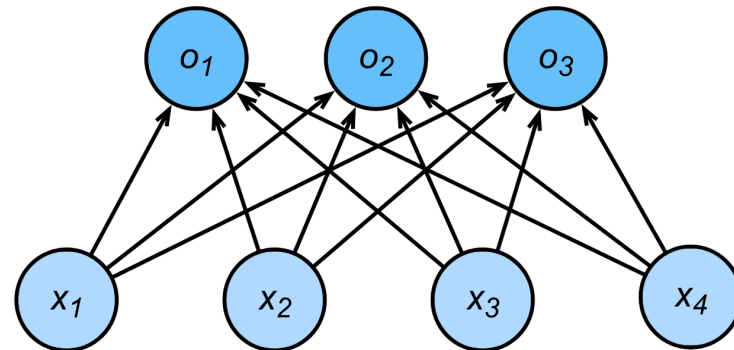
Regression

- Single continuous output
- Natural scale in $\mathbb{R}$
- Loss given e.g. in terms of difference $y - f(x)$



Classification

- Multiple classes, typically multiple outputs
- Score *should* reflect confidence ...



From Regression to Multi-class Classification

Square Loss

- One hot encoding per class

$$\mathbf{y} = [y_1, y_2, \dots, y_n]^\top$$

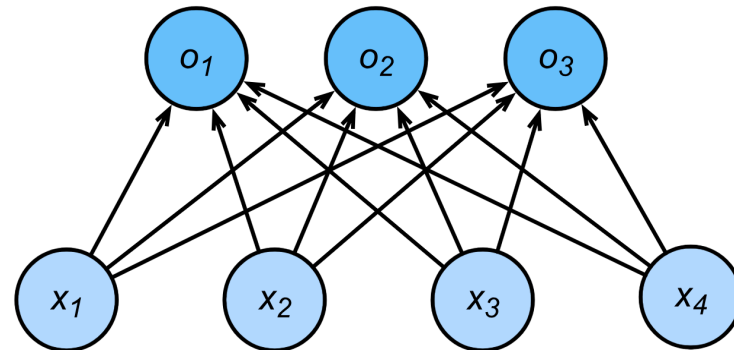
$$y_i = \begin{cases} 1 & \text{if } i = y \\ 0 & \text{otherwise} \end{cases}$$

- Train with squared loss
- Then Largest output wins

$$\hat{y} = \underset{i}{\operatorname{argmax}} o_i$$

Classification

- Multiple classes, typically multiple outputs
- Score *should* reflect confidence ...



- Negative log-likelihood (for given label y)

$$-\log p(y | o) = \log \sum_i \exp(o_i) - o_y$$

- Cross-Entropy Loss (for probability distribution)

$$\mathbf{y} = 0, 0, 0, 1_y, \dots, 0$$

$$l(\mathbf{y}, o) = \log \sum_i \exp(o_i) - \mathbf{y}^\top o$$

- **Gradient**

Difference between true
and estimated probability

$$\partial_o l(\mathbf{y}, o) = \frac{\exp(o)}{\sum_i \exp(o_i)} - \mathbf{y}^\top$$